一、摘要

NFOCC 是面向边缘 NPU（RK3588 / Axera M57）的多相机鱼眼近场感知系统： 4 路环视鱼眼 + 运动双目，覆盖 0.5–5 m，输出稠密点云与 2D 高程柱。

当前状态一句话概括：**链路已打通，精度未证实；防御代码已写好，主链未接入。**

前半句的意思是端到端能跑出点云，但”点有多准”这件事还没有可信的测量—— 唯一测精度的 `depth_accuracy` 用例失败着，而失败的原因在测试场景而不在算法。

后半句是更容易被误判的地方。D1、D3、σ 传播已在主链里跑；D2、D4、双基线 状态机的函数写好了、单元测试也过了，但 `pipeline.cpp` 一处都没调用。 **单元测试全绿证明的是函数正确，不是系统在用这些防御。**

关键数字

项	值	说明
代码规模	约 7900 行	`cpp/src` + `cpp/include`
单帧耗时	35.6 ms	单相机串行，6 视口需并行
稳态内存	162 MB	预算 291 MB，余量 44%
测试	17/18 通过	唯一失败 `depth_accuracy`
文档	40 篇全绿	`tools/docs_check.py`
真实点数	195 点/帧	开启 D1 掩码后（关闭时 661）

二、开发进展看板



2.1 已完成（9 项）

相机模型、视口 LUT、SGBM 视差、前向径向测距、三角化与体素融合、 车身掩码 D1、FoE 锥过滤 D3、σ 传播与落库、ZMQ 零拷贝、 网格哈希与 DBSCAN 聚类、动静分离、里程计积分。

这几项的判据都是实测的，不是”能编译”：

- 前向径向测距误差 0.000 mm（射线最小二乘的解析解，合成场景）
- 网格哈希 150k 点 DBSCAN 45 ms（均匀分布下实测）
- ZMQ 零拷贝：`nf_up_parse` 返回的指针确实落在消息内部
- 车身掩码负对照：开启 195 点、关闭 661 点，证明掩码确实在剔点

2.2 部分完成（3 项）

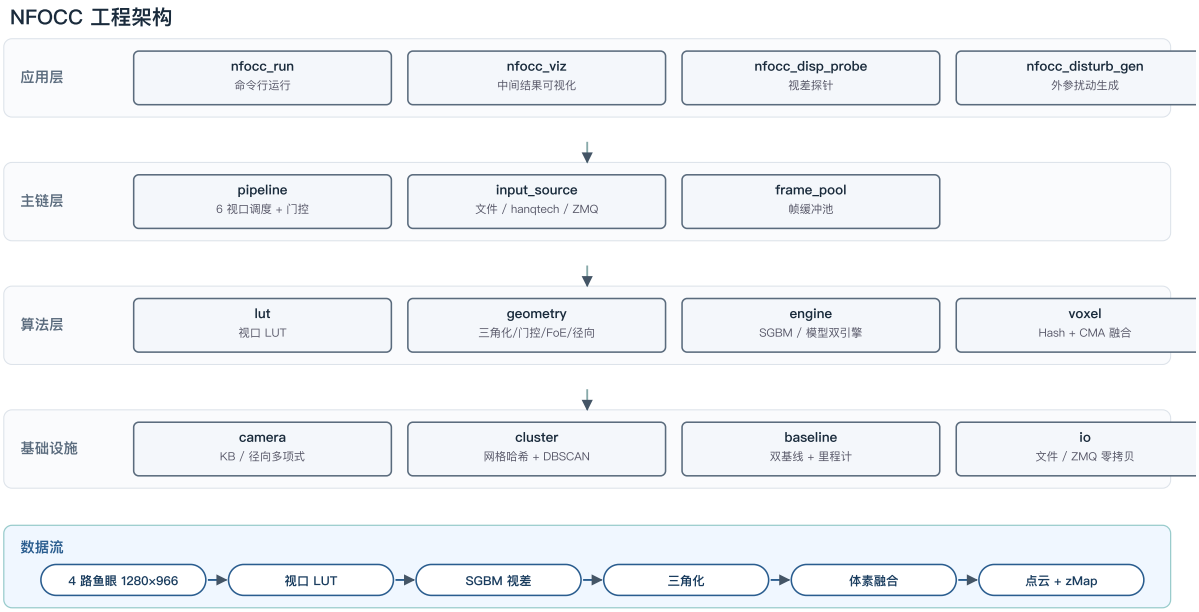
这类最容易被误报为完成。

项	代码	卡在哪
D2 双向一致性	8 用例通过	需引擎跑两次匹配，耗时 1.8 倍。是否值得要看精度收益
D4 交叉验证	函数已实现	依赖双基线晋升，而晋升依赖真实定位数据
双基线状态机	6 用例通过	`pipeline.cpp` 仍用旧的 `nf_gate_decide`

2.3 未做（5 项）

在线自校准（方案已定）、精度达标 150 mm@5 m、aarch64 上板、 Neon/SIMD 优化、RKNN 模型后端。

三、工程架构



3.1 四层结构

应用层是四个命令行工具，各自独立可跑。`nfocc_run` 做端到端， `nfocc_viz` 出中间结果图（Y 通道 / 视口 / 视差 / BEV / 掩码 / 聚类六类）， `nfocc_disp_probe` 是视差级探针（绕开三角化和体素，直接比 SGBM 输出与真值）， `nfocc_disturb_gen` 生成外参扰动的仿真数据。

主链层负责调度。`pipeline` 管 6 个视口的并行与门控， `input_source` 抽象三种数据源（文件 / hanqtech / ZMQ）。

算法层是核心计算。`lut` 建视口映射表， `geometry` 做三角化与各类判据， `engine` 是 SGBM 与模型双引擎的抽象， `voxel` 做融合。

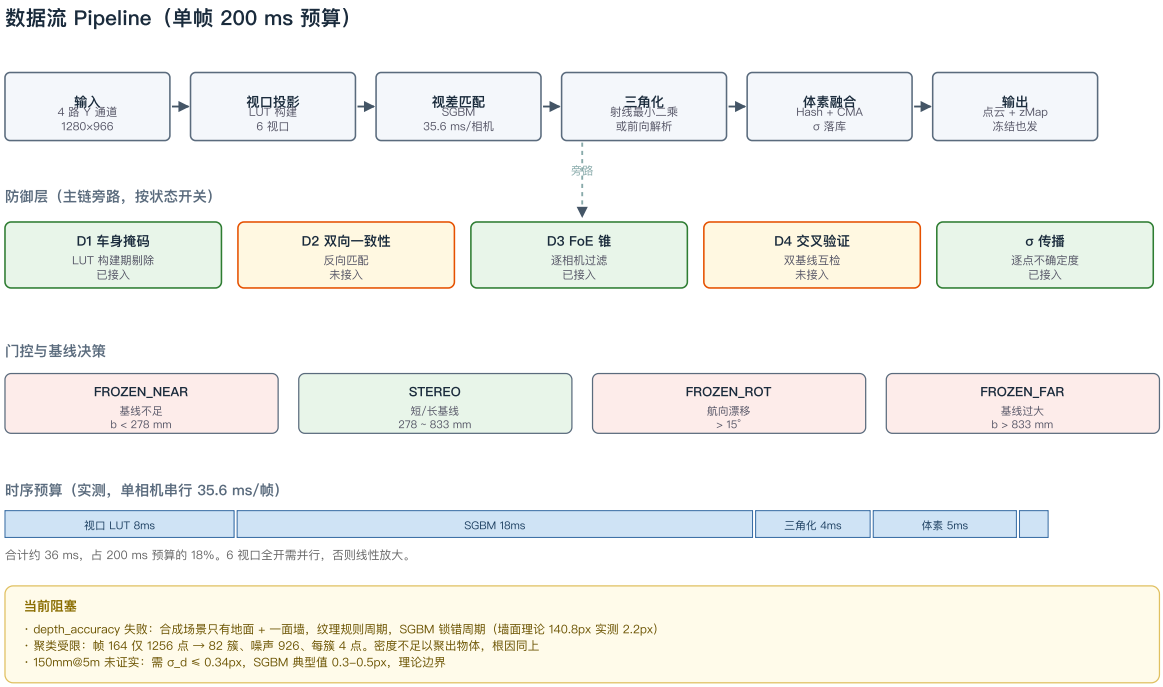
基础设施提供相机模型、空间索引、基线状态机、IO。

3.2 分层的两个约束

核心库零第三方依赖。 `nfocc_core` 的 `ldd` 里不得出现 OpenCV / ZeroMQ / onnxruntime，这条由 `scripts/build.sh core` 验证。Eigen 是 header-only 且 只用于纯计算，不算链接依赖。

核心库禁 STL 容器与异常。 `-fno-exceptions -fno-rtti`，缓冲全用 malloc。 这不是洁癖——板端不允许异常传播，且 STL 容器的分配行为不可控。

四、数据流 Pipeline



4.1 单帧流程

1. **输入**：4 路 Y 通道 1280×966（hanqtech 实拍尺寸）
2. **视口投影**：按 LUT 把原图重采样到 6 个虚拟相机视口
3. **视差匹配**：SGBM，35.6 ms/相机
4. **三角化**：射线最小二乘，或前向平移时用径向膨胀解析解
5. **体素融合**：Hash + CMA，σ 一并落库
6. **输出**：点云 + zMap，冻结时也发（下游据 `is_valid` 判断）

4.2 防御层是旁路

D1、D2、D3、D4、σ 都挂在主链旁侧，按状态开关。当前只有 D1、D3、σ 接上。

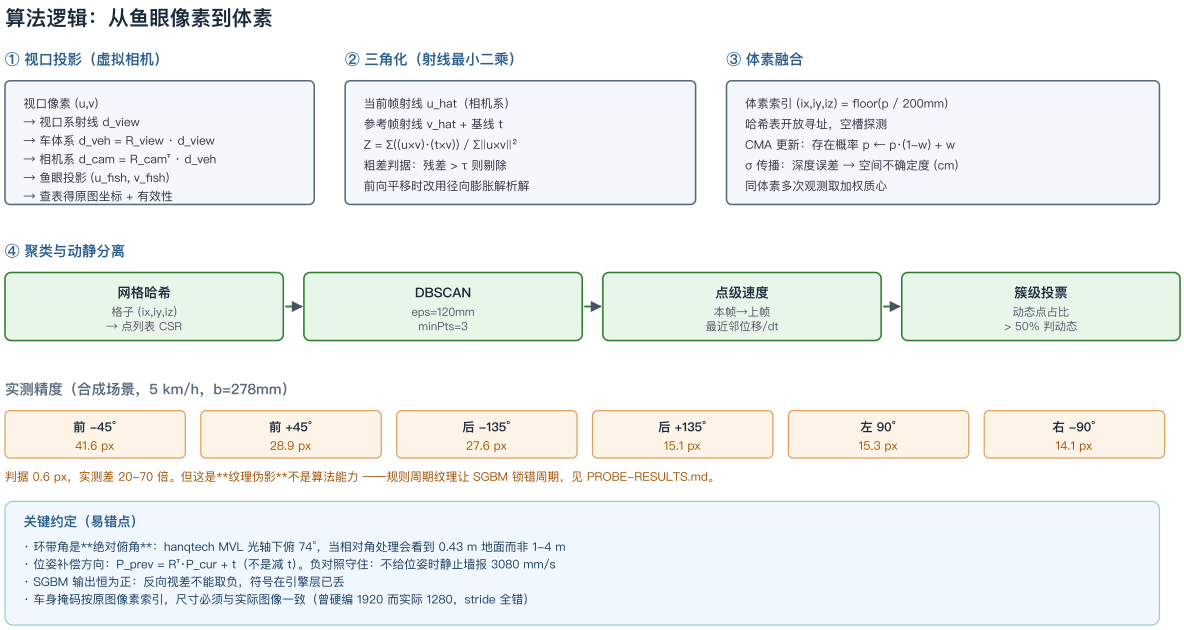
D1 的特殊之处：它在 **LUT 构建期**生效，不是逐帧计算。掩码由 `nf_mask_build_vehicle` 把车体矩形投影到每路原图生成，只在 `nf_pipeline_create` 时算一次，所以逐帧耗时不变（实测 21.3 ms 开与不开相同）。

4.3 时序

实测 35.6 ms/帧（单相机串行）。占 200 ms 预算的 18%。

但这是单相机的数。 6 视口全开若并行，仍是 35.6 ms；若串行则 214 ms， 超预算。所以并行不是优化项，是达标的必要条件。

五、算法逻辑



5.1 视口投影

虚拟相机 pitch=0、roll=0，只有 yaw。真实外参只决定”这个车系方向落在 哪张鱼眼的哪个像素”，不参与视口朝向。

视口尺寸不是随便定的：

视口	尺寸	水平张角	f (px/rad)
左右	1024 × 475	150°	137.2
前后	590 × 1024	90°	295.0

前后用 90° 而非 150°，因为 f 直接决定精度：同样基线 278 mm，f=295 在 5 m 处视差 16.4 px，f=137 只有 7.6 px。

前后各两个（±45°）是为了直行时获得**横向基线**——纯前向平移时单个前向 视口的横向基线为 0，视差深度在结构上消失。

5.2 三角化

标准做法是射线最小二乘：

$$Z = \frac{\Sigma\,((u \times v) \cdot (t \times v))}{\Sigma\,\|u \times v\|^2}$$

但**纯前向平移时这个式子退化**：u 和 v 几乎平行，叉积接近 0，数值不稳。 这也是 SGBM 在前向工况下失效的同一根源（像移沿极线方向，而 SGBM 只搜水平）。

正解是径向膨胀：前向平移时，同一点在两帧的像点到 FoE 的距离比等于深度比。 实测误差 0.000 mm（解析解）。

5.3 体素融合

体素 200 mm，哈希表开放寻址。CMA 更新存在概率：

$$p \leqslant \text{ftarrow } p \cdot (1 - w) + w$$

w 由观测的 σ 决定，σ 大的观测权重低。这样远处稀疏的观测不会污染近处 密集观测的结果。

5.4 聚类与动静分离

用**纯网格哈希**而不是树结构。这不是偷懒——八叉树上试了六版全部失败：

版本	结构	耗时	失败原因
v1	固定叶 200mm	857 ms	稠密区单叶 586 点退化
v2	自适应细分 + 全遍历	3281 ms	25325 叶全扫
v3	自适应 + x 向二分	417 ms	候选区间仍宽
v4	自适应 + 逐层哈希	4058 ms	每层扫 2197 格 × 10 层
v5	自适应 + 单层哈希	查不到	叶跨多深度，粗叶无索引
v6	统一深度 + 单层哈希	段错误	CSR/节点编号/哈希不一致

版本	结构	耗时	失败原因
v7	纯网格哈希	45 ms	—

前六版的共同失败点：**需要维护多个数据结构之间的一致性**。任何一处不同步 就表现为”查不到”或”段错误”，症状离根因很远。纯网格哈希只有一个结构 `(ix,iy,iz) → 点列表`，逻辑上不可能不一致。

格边长取 120 mm（等于 eps），查询恰好覆盖 3×3×3 = 27 格，是最小安全覆盖。

动静分离分两层：点级速度（把本帧点变换到上帧坐标系找最近邻）和 簇级投票（一簇内动态点占比超 50% 则整簇判动态）。

5.5 精度实测

视口	样本数	中位误差	偏置
前 −45°	107971	41.6 px	−22.7
前 +45°	170608	28.9 px	−30.3
后 −135°	59561	27.6 px	−20.1
后 +135°	69485	15.1 px	−0.2
左 90°	91661	15.3 px	+6.0
右 −90°	51424	14.1 px	+0.7

判据是 0.6 px，实测差 20—70 倍。

但这个数不能用来判断 SGBM 的能力。 偏置全为负说明是系统性低估而非 随机噪声，典型例子：墙面上理论视差 140.8 px，SGBM 给出 2.2 px。原因是 合成场景只有地面加一面墙，纹理是规则周期函数，SGBM 锁错周期。

六、真实数据各阶段

① Y 通道灰度图（BT.601，Y=0.299R+0.587G+0.114B）

原图 1280x966，取自 hanqtech 的 RGB PNG；后续所有匹配都在 Y 通道上做

01_y_FV.png

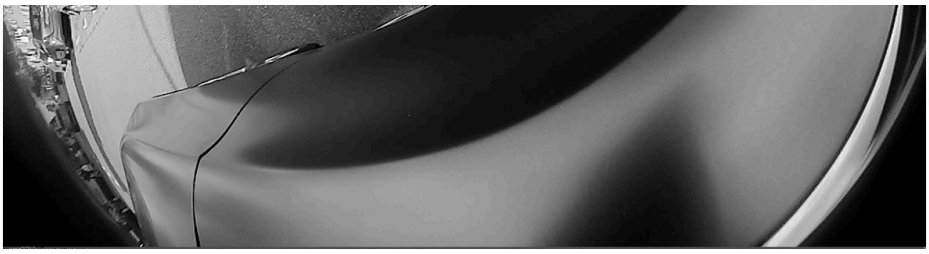


01_y_MVL.png



01_y_MVR.png





01_y_RV.png



上图：4 路鱼眼 Y 通道原图。

② 视口投影（当前帧，6 视口，640x297，f=85.74，红色=无效）

虚拟相机 pitch=0/roll=0，水平 150°/垂直 120°，yaw 按 6 方向；真实外参只决定采样到哪张鱼眼

02_view_FV_p-45.png



02_view_FV_p+45.png



02_view_RV_p-45.png



02_view_RV_p+45.png

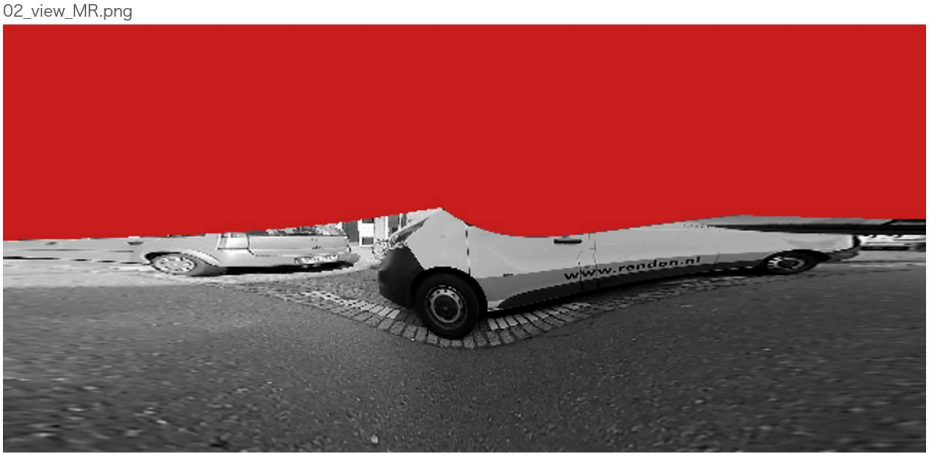


02_view_ML.png





02_view_MR.png

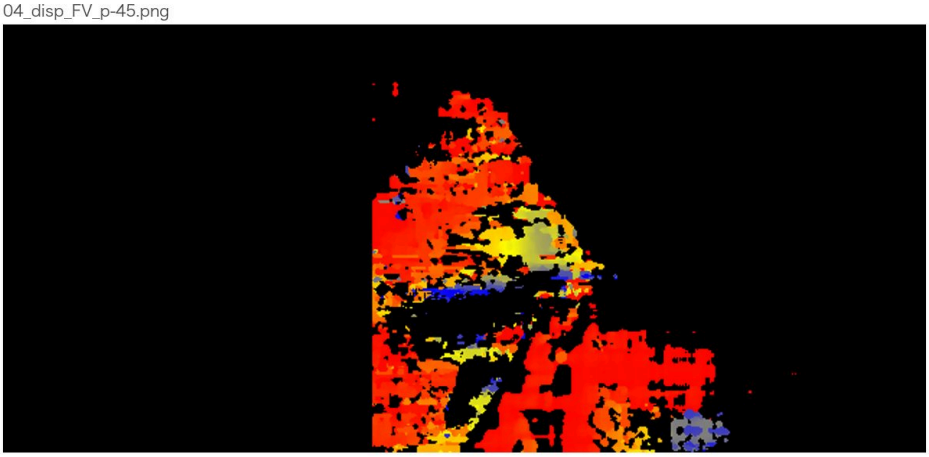


上图：视口投影结果（6 个虚拟相机）。

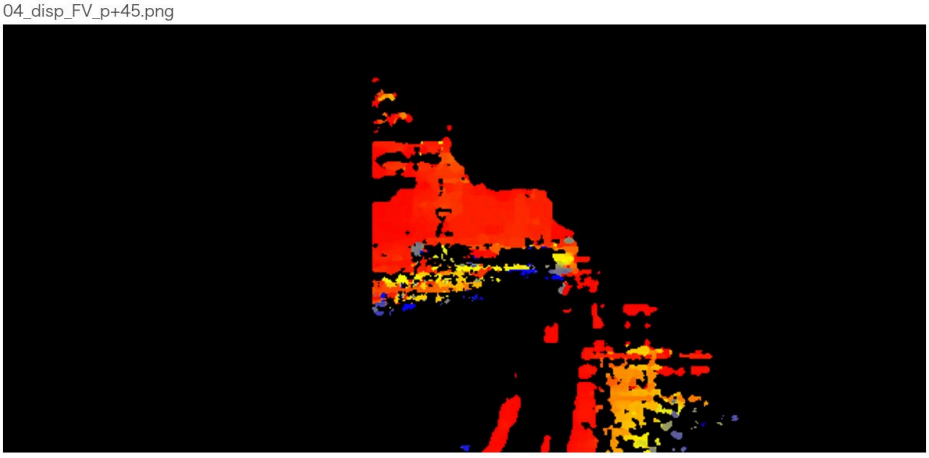
④ 视差图（6 视口，红=近，蓝=远，黑=无效）

SGBM，numDisp=256，d_max=100.9px；有效像素占比 13-19%

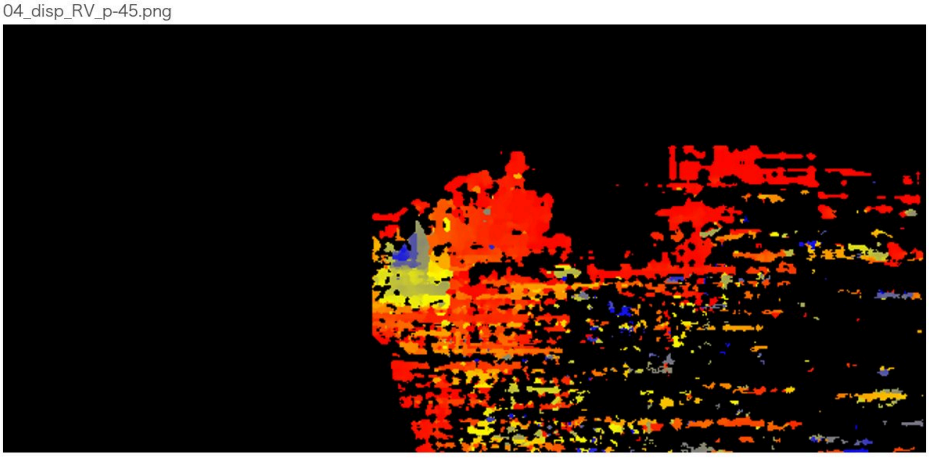
04_disp_FV_p-45.png



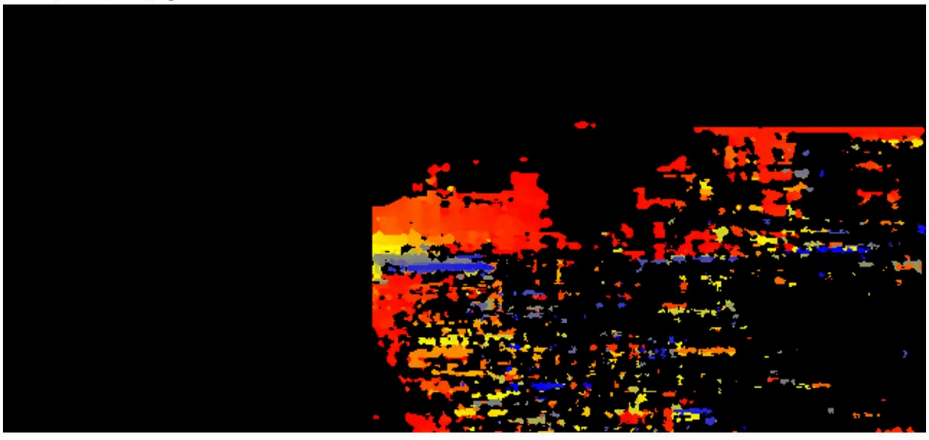
04_disp_FV_p+45.png



04_disp_RV_p-45.png

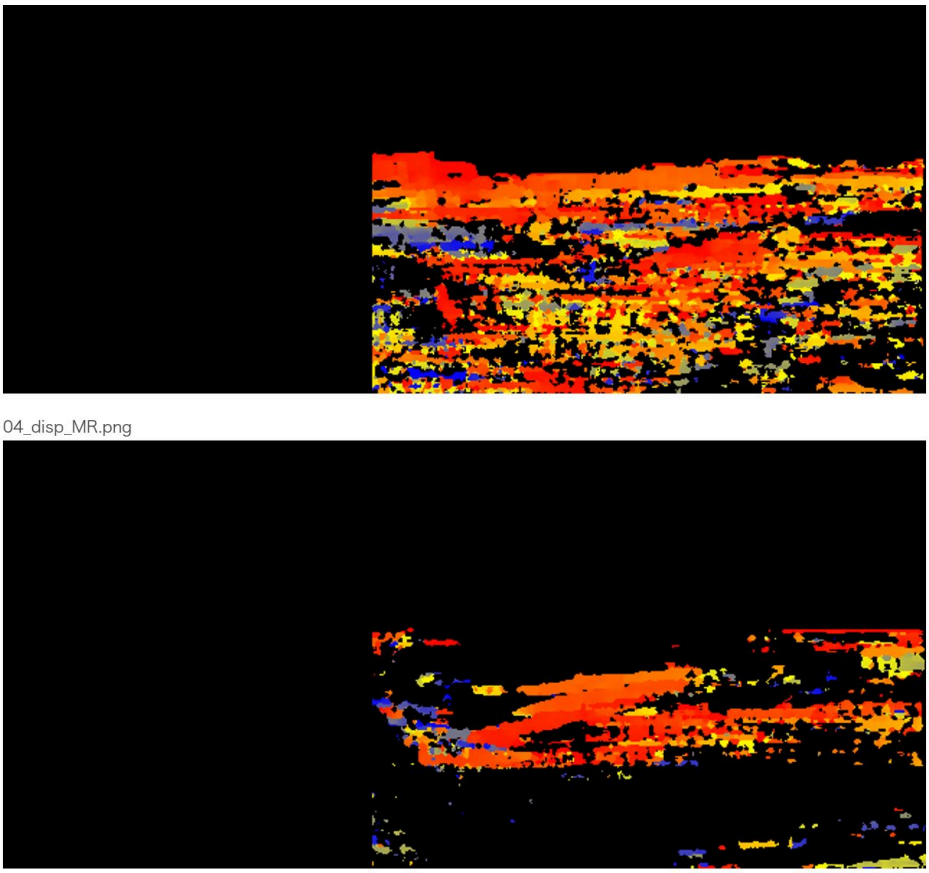


04_disp_RV_p+45.png

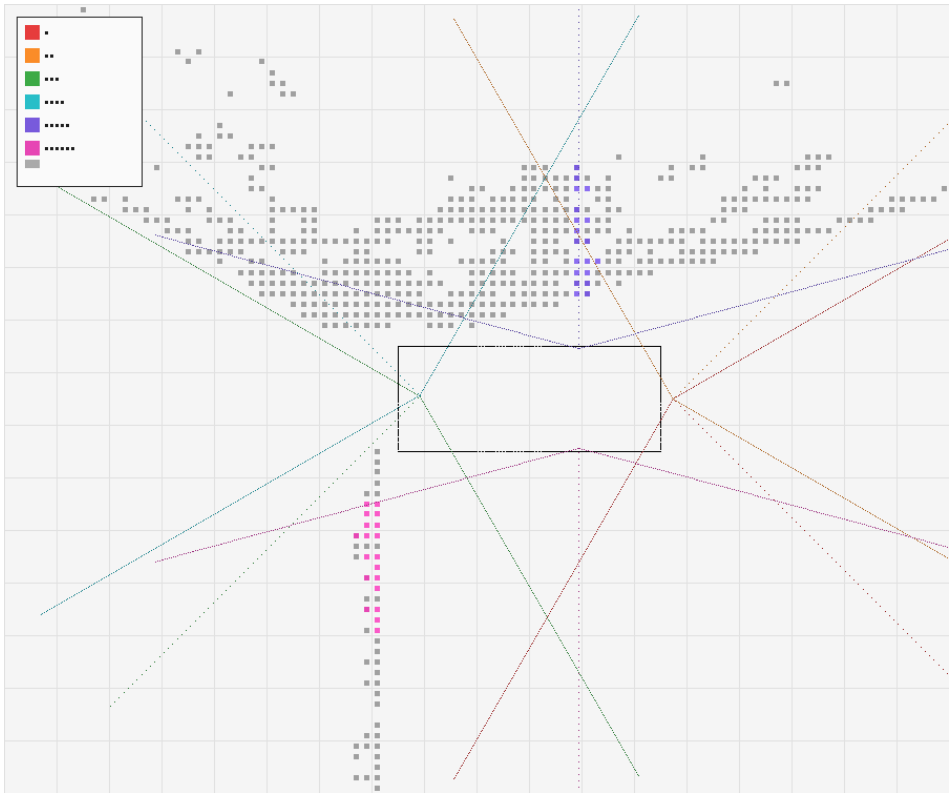


04_disp_ML.png

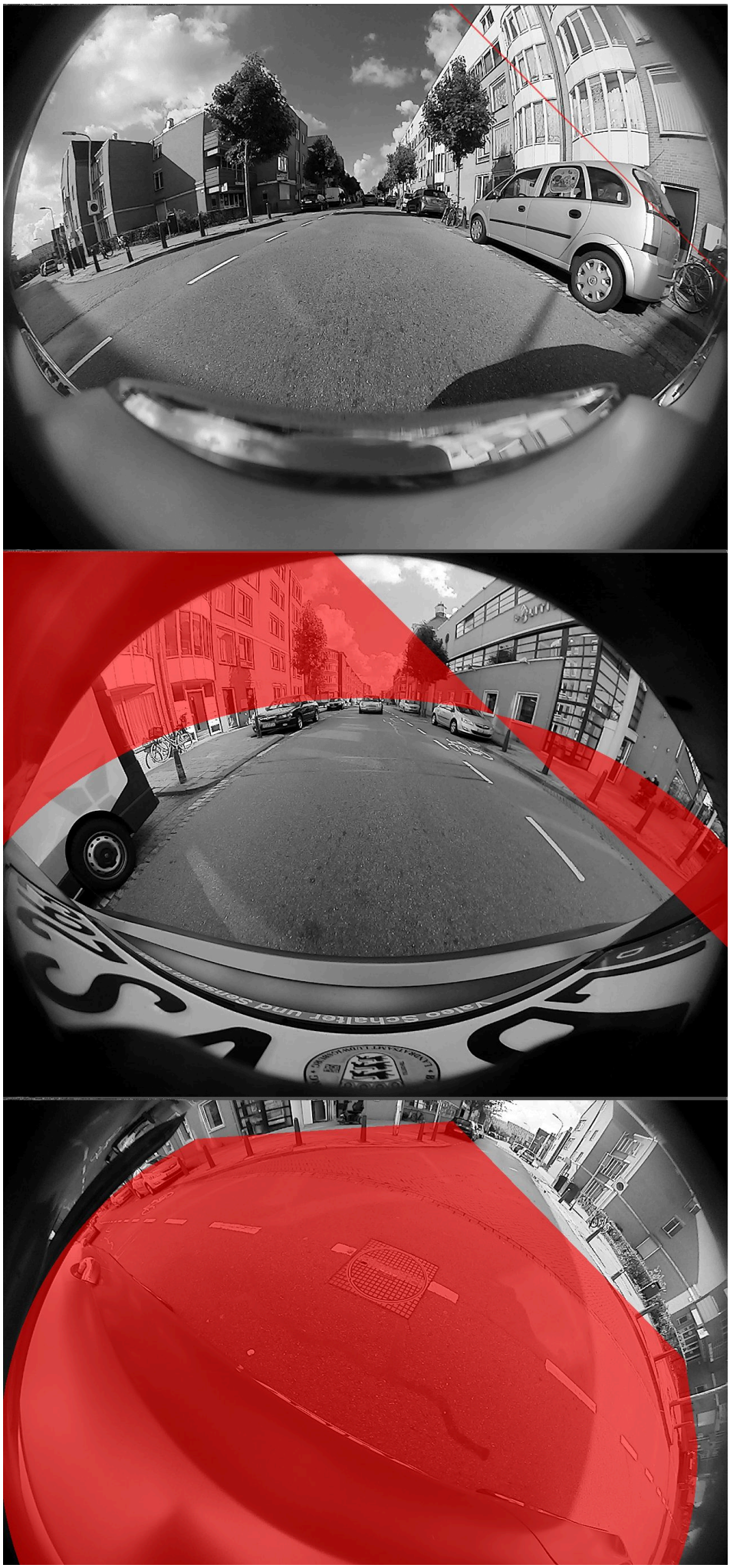


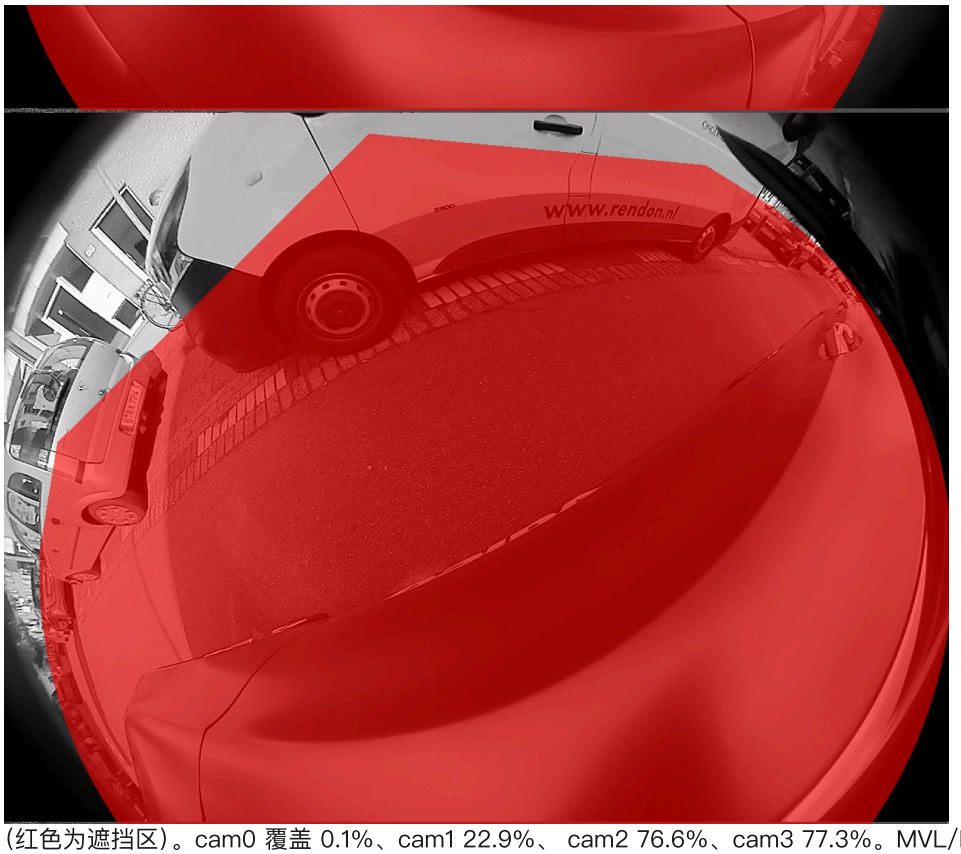


上图：SGBM 视差图。

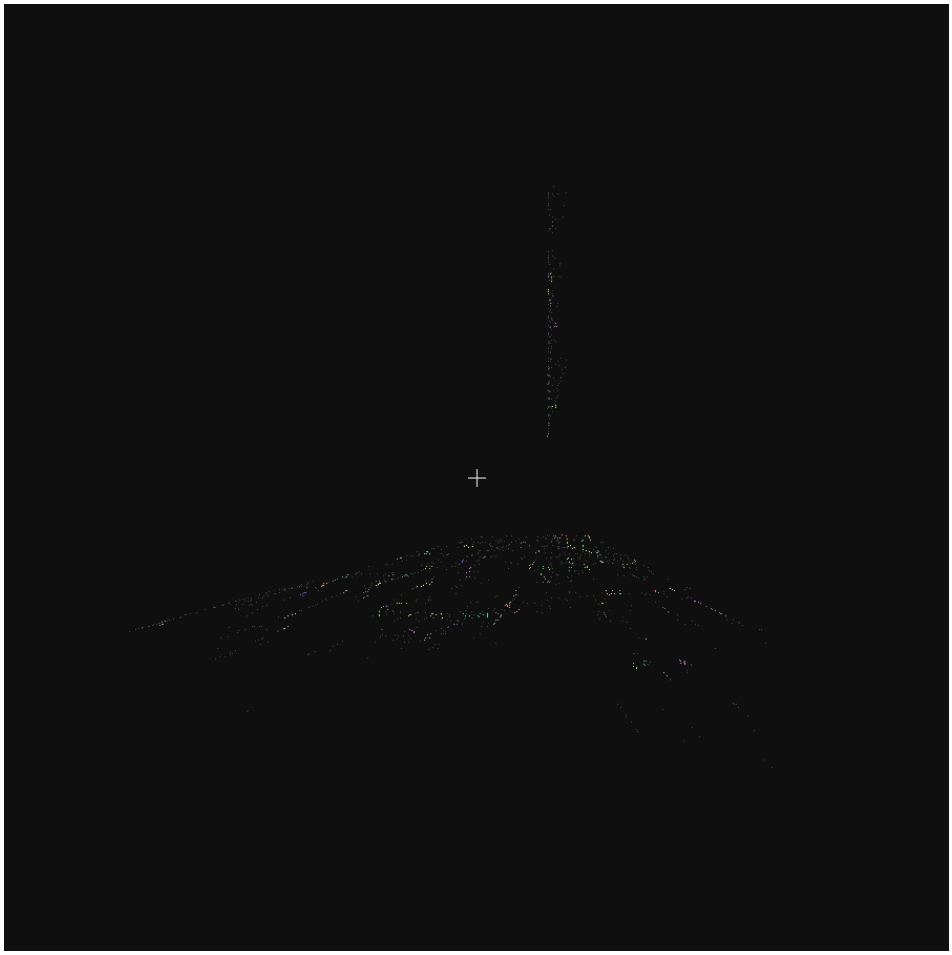


上图：体素分层 BEV。





上图：D1 车身掩码（红色为遮挡区）。cam0 覆盖 0.1%、cam1 22.9%、 cam2 76.6%、cam3 77.3%。MVL/MVR 覆盖七成是几何必然——光轴下俯 74°、 相机贴在车身边上。



上图：聚类结果 BEV（每簇一色，噪声灰，自车白十字）。

这张图值得单独说明：1256 个点聚出 82 簇，噪声 926 个，平均每簇 4 个点。 一辆车在 5 m 处应占约 50 个体素，而整帧才 1256 点。

这不是聚类算法的问题（算法正确性由 10 条用例守着，含”孤立点必须判噪声”这类判据），而是**点密度不足**。根因又回到精度：σ_d 14–42 px 导致大量 点被剔除。

七、已知问题与边界

7.1 depth_accuracy 失败

合成场景纹理是规则周期函数，SGBM 锁错周期。这是**场景缺陷不是算法能力**。

修法明确：给场景加随机相位纹理后重跑探针。这个数才决定 150 mm 目标 能不能立住。

7.2 150 mm @ 5 m 未证实

5 km/h 时理论需要 σ_d ≤ 0.34 px，而 SGBM 典型值 0.3–0.5 px。这是理论边界， 不是调参能解决的。分距离段的验收标准见 `SPEC-DEPTH-ACCEPTANCE.md`：

距离段	前后视口	左右视口
0.5–2 m	< 150 mm	< 150 mm
2–3 m	< 150 mm	< 200 mm
3–4 m	< 200 mm	< 320 mm
4–5 m	< 320 mm	< 500 mm

前提是 σ_d ≤ 0.5 px 已证实，而当前实测 14–42 px。前提没成立，这些数字 就只是理论值。

7.3 外部数据缺失

缺什么	影响
点云真值	无法做真实精度验收
高精定位 / IMU	双基线只能用 yaw_rate 积分降级，打滑时失效
基线真值	2% 里程误差是假设，未验证
训练权重	模型引擎通路通，无权重可跑
实车轮廓	车身掩码用 5000×2000 矩形占位，会过度屏蔽

7.4 一个已修掉但值得记录的冲突

`nf_up_parse` 原先硬校验 1920×1536，而 hanqtech 实拍是 1280×966。 ZMQ 通路接上真实数据后每一帧都会被判格式错误直接丢弃。

测试没发现是因为测试数据是按 ICD 尺寸造的。 这种”测试通过但不能用”的情况，只有真的接上真数据才会暴露。

现已改为白名单（默认容纳两种尺寸，可配置），并由 `up_parse_accepts_both_sizes` 用例守着。

八、快速上手

```
cd cpp
./scripts/build.sh host          # 构建 + 测试
ctest --test-dir build/host -E depth_accuracy   # 排除已知失败

# 真实数据
./build/host/nfocc_run --hanqtech ../datasets/fisheye --ws-frame 164 --frames 3 --stats
./build/host/nfocc_viz ../datasets/fisheye 164 ../viz

# 验证
python3 ../tools/docs_check.py
../scripts/verify_docs.sh
../scripts/verify_zmq.sh
```

完整说明见仓库根目录 `README.md`。

附录：文档索引

主题	文档
研发计划	`ROADMAP.md`
精度决策	`DECISION-DEPTH-ACCURACY.md`
验收标准	`SPEC-DEPTH-ACCEPTANCE.md`
坐标系（唯一权威）	`SPEC-CONVENTION.md`
真实数据接入	`SPEC-DATASET.md`
聚类实现	`CLUSTERING-IMPLEMENTATION.md`
防御接入现状	`PLAN-DEFENSE-INTEGRATION.md`
构建调试	`BUILD-CROSSPLATFORM.md`
探针结果	`PROBE-RESULTS.md`